

Cache 接入 CPU 技术细节

CPU 实现的改动

旧有 CPU 实现与 Cache 接入 CPU 存在一定冲突：

- 数据存储器零延迟，访存结果可以在同一个时钟周期得到；在接入 Cache 之后，CPU 需要**处理 Cache 的 miss, ready 信号**，并据此作出停顿/恢复流水线的响应
- store 指令的字节对齐位于 CPU 模块内；在接入 Cache 后，更好的做法是**将字节对齐下放到 Cache 模块中进行**，以简化 CPU 的操作（只向 Cache 暴露出完整的写内存意图），把写访存的实现细节交给 Cache 内部完成

在 **CPU 模块** `cpu.v` 中的更改

- 撤销原有 CPU 与数据存储器的直接交互端口，改为向 Cache 端口发起内存交互请求
- CPU 根据 Cache 传出的 miss, ready 信号相应作出停顿、恢复流水响应

在数据内存控制模块 `data_mem_ctrl.v` 中的更改

- 执行 store 指令时，不再将写入数据与字中已有数据合并后作为输出 `wdata_out`，而是只向模块外提供**按字节对齐的写入数据** `wdata_out` 和字写入掩码 `we_mask`

Cache 实现的改动

- Cache 新增输出端口 `miss, ready`，用于向 CPU 同步访存状态，在 Cache Miss 处理时让 CPU 相应作出**流水线停顿**
- Cache Hit 时的实现逻辑从写回改为**写直达**，相应添加变量 `write_line_buf`，以便 PDU 模块可以在调试时**随时获取主存的理论最新状态**
- 状态机重新划分为 IDLE, LOOKUP, MISS_READ, REFILL, W_THROUGH 五个状态
 - IDLE 状态，Cache 在接收到 CPU 发出的读/写信号 `r/w_req` 时，缓存同时传入的访存地址 `addr`、写掩码 `w_mask`、待写入数据 `w_data`
 - LOOKUP 状态，Cache 利用访存地址 Tag 段判断 Cache Hit
 - 如果是 read hit，则截取目标字节直接反馈给 CPU 并跳转到 IDLE 状态
 - 如果是 write hit，则跳转到 W_THROUGH 状态
 - 如果 Cache Miss，则跳转到 MISS_READ 状态
 - MISS_READ 状态，Cache 向主存发出读取请求，在主存就绪后将读出数据锁存，并跳转到 REFILL 状态
 - REFILL 状态，Cache 将主存读出数据写入由 LRU 替换策略指定的 Way 中，同时**将目标字节前递给 CPU**，跳转到 IDLE 状态

- `W_THROUGH` 状态，Cache 将命中的 Cache Line 使用主存写入数据覆写后**向主存发出写请求**，在主存就绪后跳转到 `IDLE` 状态

接入 Cache 后 CPU 的访存指令执行路径

- **IF 段**：根据 PC 从指令存储器中读取访存指令
- **ID 段**：控制单元对指令进行译码，生成访存控制信号（`mem_read / mem_write`），并读取基址寄存器数据、生成立即数偏移量
- **EX 段**：ALU 将基址数据与立即数偏移量相加，计算出目标内存物理地址
- **MEM 段**
 - `data_mem_ctrl` 根据目标地址后两位与指令操作类型（B/H/W），对待写入的 Store 数据进行字节对齐并生成对应的字节屏蔽掩码 `we_mask`
 - CPU 向 Cache 模块单次发出访存请求，并利用 `dcache_req_active` 防止流水线停顿时陷入重复发请求死循环
 - 若 Cache Miss 或未就绪（`miss=1` 或 `ready=0` 为），CPU 内部产生 `dcache_wait = 1` 停顿流水线
 - 当 Cache 命中或从主存 Refill 完成、`ready` 置 1 后，流水线恢复运行。若是 Load 请求，`data_mem_ctrl` 按照指令格式（位宽与是否有符号）完成字节截取与对应的扩展处理，输出结果
- **WB 段**：若是 Load 指令，将读出数据写入对应的通用目标寄存器

分支预测模块技术细节

局部预测器的结构与工作流程

- 核心组件包括 BHT(Branch History Table)、PHT(Pattern History Table)
- BHT 以 PC 的部分位 `pred_pc_idx` 作为索引，“低位近期”地存储过去 `N=LOCAL_HIST_W` 次跳转历史
- PHT 以 BHT 过去 `N` 次跳转历史 `pred_hist` 作为索引，存储预测跳转情况的 2-bits 状态机
 - `pred_hist` 同时传出模块，进入分支预测段间寄存器随流水线向下流动
 - 状态机值的含义：00-强不跳转，01-弱不跳转，10-弱跳转，11-强跳转
 - 取其中高位作为预测输出，**如果高位为 1 则预测当次会跳转**，否则预测当次不跳转
- 在 EX 段更新 BHT、PHT 的值
 - 从 `train_hist_snapshot` 中获取同一条指令在预测时的 BHT 状态（即 `pred_hist`）
 - 使用传入的 `train_pc_idx`，`train_taken` 在 BHT 中更新该指令的跳转历史
 - 如果执行当次指令时实际发生跳转 `taken=1`，则**将状态机的值加 1**直到达到 11；如果实际未发生跳转，则**将状态机的值减 1**直到达到 00

```

module local_predictor #(
    parameter PC_IDX_W = 8,          // PC Index 段位宽, BHT 共 2^PC_IDX_W 项
    parameter LOCAL_HIST_W = 6      // 局部历史位宽, PHT 共 2^LOCAL_HIST_W 项
)(
    input wire clk,                  // 时钟信号
    input wire rst,                  // 复位信号
    input wire en,                   // 使能信号

    input wire [PC_IDX_W-1:0] pred_pc_idx,    // 预测阶段 (IF 段) 指令的 pc_idx
    output wire [LOCAL_HIST_W-1:0] pred_hist,  // IF 段预测时的 bht[pc_idx] 快照
    output wire pred_taken,                // 局部预测器预测的跳转方向 (1-跳转, 0-不跳转)

    input wire train_valid,             // 训练阶段 (EX 段) 的有效信号, valid 为 1 时有效
    input wire [PC_IDX_W-1:0] train_pc_idx, // EX 段指令的 pc_idx
    input wire [LOCAL_HIST_W-1:0] train_hist_snapshot, // EX 段指令在预测阶段的 bht[pc_idx] 快照
    input wire train_taken               // EX 段指令实际跳转情况
);

localparam integer BHT_ENTRIES = (1 << PC_IDX_W); // BHT 容量
localparam integer PHT_ENTRIES = (1 << LOCAL_HIST_W); // PHT 容量

reg [LOCAL_HIST_W-1:0] bht [0:BHT_ENTRIES-1]; // 分支历史表, 以 pc_idx 为索引, 存储对应:
// 模式历史表, 以 BHT 对 pc_idx 的映射结果为索引, 存储 2-bit 状态机 (跳转倾向)
// 00-强不跳转, 01-弱不跳转, 10-弱跳转, 11-强跳转
reg [1:0] pht [0:PHT_ENTRIES-1];

integer i;

// 2-bit 状态机更新函数
function [1:0] sat_update;
    input [1:0] cur; // 当前状态
    input taken; // 当前实际跳转情况
    begin
        sat_update = cur;
        if (taken) begin
            if (cur != 2'b11) begin
                sat_update = cur + 2'b01; // 实际发生跳转, 计数器加 1 (2'b11 封顶)
            end
        end
        else begin
            if (cur != 2'b00) begin
                sat_update = cur - 2'b01; // 实际未发生跳转, 计数器减 1 (2'b00 封底)
            end
        end
    end
end

```

```

    end
endfunction

// 跳转预测结果
assign pred_hist = bht[pred_pc_idx];      // 根据当前 pc_idx 读出跳转局部历史
assign pred_taken = pht[pred_hist][1];    // 使用读出的局部历史查找 PHT，取计数器高位作为预测结果

always @(posedge clk) begin
    if (rst) begin
        // 复位时，清空 BHT
        for (i = 0; i < BHT_ENTRIES; i = i + 1) begin
            bht[i] <= {LOCAL_HIST_W{1'b0}};
        end
        // 复位时，PHT 初始化为弱不跳转
        for (i = 0; i < PHT_ENTRIES; i = i + 1) begin
            pht[i] <= 2'b01;
        end
    end
    else if (en && train_valid) begin
        // 指令执行到 EX 段时，pred_pc_idx 可能变化，不能取出当前指令对应的 bht 条目
        // 因此，使用 IF 段预测时外传的 bht[pc_idx] 快照更新 PHT
        pht[train_hist_snapshot] <= sat_update(pht[train_hist_snapshot], train_taken);
        // 更新 BHT：左移并将当前的实际跳转结果存入历史最低位
        bht[train_pc_idx] <= {bht[train_pc_idx][LOCAL_HIST_W-2:0], train_taken};
    end
end
endmodule

```

全局预测器的结构与工作流程

- 核心组件包括 GHR(Global History Register)、PHT
- GHR “低位近期”地记录整个程序过去 GHR_W 次执行分支指令的跳转情况
- PHT 以 $\text{ghr}^{\text{pred_pc_idx}}$ 作为索引存储预测跳转情况的 2-bits 状态机
 - $\text{pred_ghr} = \text{ghr}$ 同时传出模块，进入分支预测段间寄存器随流水线向下流动
 - 通过异或操作实现**对全局跳转历史、指令位置信息的哈希映射**，从而让索引能在一定程度上同时反映二者的信息，在不大幅度扩大存储需求的情况下，**对相同全局历史不同指令、相同指令不同执行路径两种情况都可以做出差异化的预测**
- 在 EX 段更新 GHR、PHT 的值
 - 从 train_ghr_snapshot 中获取分支指令在 IF 段执行时的 GHR 状态（即 pred_ghr）
 - 使用传入的 train_pc_idx, train_taken 在 GHR 中更新该指令的跳转历史

- 如果执行当次指令时实际发生跳转 $taken=1$ ，则**将状态机的值加 1**直到达到 11；如果实际未发生跳转，则**将状态机的值减 1**直到达到 00

```

module global_predictor #(
    parameter PC_IDX_W = 8,          // PC 索引位宽
    parameter GHR_W = 8             // 全局历史寄存器位宽
)(
    input wire clk,                  // 时钟信号
    input wire rst,                  // 复位信号
    input wire en,                   // 使能信号

    input wire [PC_IDX_W-1:0] pred_pc_idx, // 预测阶段（IF 段）指令的 pc_idx
    output wire [GHR_W-1:0] pred_ghr,      // IF 段预测时的 ghr 快照
    output wire pred_taken,                // 局部预测器预测的跳转方向（1-跳转，0-不跳转）

    input wire train_valid,              // 训练阶段（EX 段）的有效信号，valid 为 1 时有效
    input wire [PC_IDX_W-1:0] train_pc_idx, // EX 段指令的 pc_idx
    input wire [GHR_W-1:0] train_ghr_snapshot, // EX 段指令在预测阶段的 ghr 快照
    input wire train_taken,              // EX 段指令实际跳转情况
);

localparam integer PHT_ENTRIES = (1 << PC_IDX_W); // PHT 容量

reg [GHR_W-1:0] ghr; // 全局历史寄存器（Global History Register）
reg [1:0] pht [0:PHT_ENTRIES-1]; // 模式历史表（2-bit 状态机）

integer i;

// 2-bit 状态机更新函数
function [1:0] sat_update;
    input [1:0] cur; // 当前状态
    input taken; // 当前实际跳转情况
    begin
        sat_update = cur;
        if (taken) begin
            if (cur != 2'b11) begin
                sat_update = cur + 2'b01; // 实际发生跳转，计数器加 1（2'b11 封顶）
            end
        end
        else begin
            if (cur != 2'b00) begin
                sat_update = cur - 2'b01; // 实际未发生跳转，计数器减 1（2'b00 封底）
            end
        end
    end
endfunction

```

```

// IF 段的索引: pc_idx 与 GHR 按位异或, 以同时包括双方信息
wire [PC_IDX_W-1:0] pred_idx = pred_pc_idx ^ ghr[PC_IDX_W-1:0];
// EX 段的索引
wire [PC_IDX_W-1:0] train_idx = train_pc_idx ^ train_ghr_snapshot[PC_IDX_W-1:0];

// 组合逻辑输出预测快照和预测结果
assign pred_ghr = ghr;
assign pred_taken = pht[pred_idx][1]; // 取对应 PHT 表项计数器的高位作为预测方向

always @(posedge clk) begin
    if (rst) begin
        ghr <= {GHR_W{1'b0}}; // 复位时清零全局历史
        for (i = 0; i < PHT_ENTRIES; i = i + 1) begin
            pht[i] <= 2'b01; // PHT 初始化为弱不跳转
        end
    end
    else if (en && train_valid) begin
        // 指令执行到 EX 段时, pred_pc_idx 可能变化, 不能取出当前指令对应的 bht 条目
        // 因此, 使用 IF 段预测时外传的 ghr 快照和 pc_idx 快照更新 PHT
        pht[train_idx] <= sat_update(pht[train_idx], train_taken);
        // 更新 GHR: 左移并将当前的实际跳转结果存入历史最低位
        ghr <= {ghr[GHR_W-2:0], train_taken};
    end
end
endmodule

```

竞争预测器工作流程

- PHT 以 `pred_pc_idx` 作为索引, 存储预测器选用倾向
 - 00-强局部, 01-弱局部, 10-弱全局, 11-强全局, 取其中高位作为预测输出
 - 在局部预测器与全局预测器预测结果不同时, **将状态机向预测正确的预测器方向转移**, 例如全局预测器正确、局部预测器错误时, 状态机值加 1

```

module choice_predictor #(
    parameter PC_IDX_W = 8          // PC 索引位宽
)(
    input wire          clk,          // 时钟信号
    input wire          rst,          // 复位信号
    input wire          en,           // 使能信号

    input wire [PC_IDX_W-1:0] pred_pc_idx, // 预测阶段读出的 PC 索引
    output wire          pred_use_global, // 预测输出: 1-全局, 0-局部

    input wire          train_valid, // 训练阶段 (EX 段) 有效信号
    input wire [PC_IDX_W-1:0] train_pc_idx, // EX 段 pc_idx
    input wire          train_local_pred, // 局部预测器作出预测情况快照
    input wire          train_global_pred, // 全局预测器作出预测情况快照
    input wire          train_actual_taken // EX 段实际跳转方向
);

localparam integer CHOICE_ENTRIES = (1 << PC_IDX_W); // 竞争表项数

reg [1:0] choice_pht [0:CHOICE_ENTRIES-1]; // 竞争表, 包含 2-bit 状态机
integer i;

// 竞争表 2-bit 状态机更新函数
function [1:0] sat_update;
    input [1:0] cur; // 当前计数值
    input      move_up; // 1-加深全局倾向, 0-加深局部倾向
    begin
        sat_update = cur;
        if (move_up) begin
            if (cur != 2'b11) begin
                sat_update = cur + 2'b01;
            end
        end
        else begin
            if (cur != 2'b00) begin
                sat_update = cur - 2'b01;
            end
        end
    end
endfunction

// 高位为 1 表示使用全局预测结果, 高位为 0 表示使用局部预测结果
assign pred_use_global = choice_pht[pred_pc_idx][1];

```

```

always @(posedge clk) begin
    if (rst) begin
        for (i = 0; i < CHOICE_ENTRIES; i = i + 1) begin
            choice_pht[i] <= 2'b01; // 初始化为弱局部
        end
    end
    else if (en && train_valid) begin
        if (train_local_pred != train_global_pred) begin
            // 只在局部/全局预测结果不同时更新选择倾向
            if (train_global_pred == train_actual_taken) begin
                // 全局对而局部错，增加全局倾向
                choice_pht[train_pc_idx] <= sat_update(choice_pht[train_pc_idx], 1'b1);
            end
            else if (train_local_pred == train_actual_taken) begin
                // 局部对而全局错，增加局部倾向
                choice_pht[train_pc_idx] <= sat_update(choice_pht[train_pc_idx], 1'b0);
            end
        end
    end
end
endmodule

```

分支预测系统整体 workflow

- 核心组件包括 BTB(Branch Target Buffer)、RAS(Return Address Stack)
- 获得 IF 段的指令地址 if_pc 后，根据截取出的 if_pc_idx, if_pc_tag 段到 BTB 中查找指令
- 若未命中，预测系统不工作，if_pred_valid 置 0；若命中，BTB 返回指令类型 if_btb_kind 和指令跳转地址 if_btb_target
- 根据**指令类型**分三条路径进一步处理
 - 若属于条件分支指令 KIND_BRANCH，则根据竞争预测器当前状态选择采用局部预测器/全局预测器的跳转预测结果；若预测跳转，则采用 BTB 缓存的跳转地址 if_btb_target
 - 若属于直接跳转 KIND_JAL，则采用 BTB 缓存的跳转地址 if_btb_target 作为目标地址
 - 若属于间接跳转 KIND_JALR，则在 RAS 非空情形下取 RAS 栈顶地址，若 RAS 为空则取 BTB 缓存的跳转地址 if_btb_target
- 在 EX 段，根据随段间寄存器下传的 IF 段预测结果等信息 if_pred_meta，**确定是否发生预测错误**
 - 类型错误 ex_kind_mismatch：自修改跳转指令在执行一次后改为非跳转指令，或混淆 if_pc_idx 相同的两条指令
 - 预测错误 ex_direction_mismatch：预测结果与实际跳转情况不同
 - 目标地址错误 ex_target_mismatch：预测的跳转地址与实际跳转地址不符
- 若发生预测错误，则将 redirect_valid 信号置 1，清除处于 IF, ID 段的指令

- 基于实际指令执行情况，修改 BTB
 - 若实际执行的确实是分支指令 `btb_update_fire=1`，则将跳转地址覆盖到对应的 BTB 单元 `btb_tag/kind/target[ex_pc_idx]` 上
 - 若实际执行的是普通指令但命中 BTB `btb_invalidate_fire=1`，则将对应 BTB 单元置为无效 `btb_valid[ex_pc_idx] <= 0`
 - 若实际执行的是普通指令且未命中 BTB，则不对 BTB 进行任何修改
- RAS 的**双栈机制**
 - RAS 分为架构栈（稳定）与投机栈（激进）
 - 在 IF 段，若预测当前指令是 JAL 指令，就直接将 `if_pc+32'd4` 压入投机 RAS；若预测当前指令是 JALR 指令，就直接弹出栈顶地址作为预测地址
 - 在 EX 段，**架构 RAS** 根据实际执行的指令进行压栈/出栈操作；若指令预测错误，投机 RAS 利用架构 RAS 的当前状态进行覆写以**排除所有错误栈操作**

分支预测性能比较

为方便分析分支预测给 CPU 带来的性能提升，在 CPU 中引入了一个**时钟周期计数器**，其具体工作流程如下：

1. **清空与初始化**：当 `rst` 或 `benchmark_clear` 有效时，计时器清零，运行状态 `benchmark_running_reg`、完成状态 `benchmark_done_reg` 和周期计数 `benchmark_cycles_reg` 均复位。接收到 `benchmark_clear` 信号时，额外发出应答信号 `benchmark_clear_ack` 供 PDU 确认计数器复位状态。
2. **启动计时**：当第一条指令进入 IF 段，满足 `benchmark_start` 条件（流水线运行、**计时器未处在运行或及时完成阶段**、PDU 已准备好进行性能采样），计时器开始运行，记为第 1 个周期，并将计时器状态设为运行中 `benchmark_running_reg=1`。
3. **运行中**：在运行状态下，只要 `global_en=1` 成立，计时器计数就会在当前时钟周期加 1，因此**计数值包含了所有的停顿时钟周期**，如 Cache Miss 的等待周期、Load-Use 数据冒险的流水线 Stall，以及分支预测失败导致的 Flush 惩罚周期等，从而能够如实反映程序的实际执行性能。
4. **计时完成**：当 EBREAK 指令在 WB 段执行完成后，满足 `benchmark_stop` 条件（计时器处在运行状态、流水线未停顿、WB 段指令有效且为停机指令），计时器停止。此时会执行最后一次计数值加 1，以计入 EBREAK 指令在 WB 段执行的时钟周期，并将计时器状态设为计时完成 `benchmark_done_reg=1`。

```

reg benchmark_running_reg;
reg benchmark_done_reg;
reg [31:0] benchmark_cycles_reg;
reg benchmark_clear_ack_reg;

wire benchmark_start = benchmark_arm && !benchmark_running_reg && !benchmark_done_reg && global_
wire benchmark_stop  = benchmark_running_reg && commit_advance && commit_WB && halt_WB;

always @(posedge clk) begin
    if (rst) begin
        benchmark_running_reg <= 1'b0;
        benchmark_done_reg <= 1'b0;
        benchmark_cycles_reg <= 32'b0;
        benchmark_clear_ack_reg <= 1'b0;
    end
    else if (benchmark_clear) begin
        // PDU 在发起 brun 前先清空旧结果; ack 是 CPU 域保持型应答, 供 TOP/CPU_ctrl 跨域回收 clear
        benchmark_running_reg <= 1'b0;
        benchmark_done_reg <= 1'b0;
        benchmark_cycles_reg <= 32'b0;
        benchmark_clear_ack_reg <= 1'b1;
    end
    else begin
        benchmark_clear_ack_reg <= 1'b0;

        if (benchmark_start) begin
            // 第一条指令被 IF 接收的周期计为第 1 个周期, 保证 Start 边界包含程序开始执行的那一拍。
            benchmark_running_reg <= 1'b1;
            benchmark_done_reg <= 1'b0;
            benchmark_cycles_reg <= 32'd1;
        end
        else if (benchmark_running_reg && global_en) begin
            if (benchmark_stop) begin
                // EBREAK 完全退休的周期也必须计入总周期数, 因此在停止时锁存 cycles + 1。
                benchmark_running_reg <= 1'b0;
                benchmark_done_reg <= 1'b1;
                benchmark_cycles_reg <= benchmark_cycles_reg + 32'd1;
            end
            else begin
                // global_en 期间每个 CPU 周期都计入, 包括 Cache miss、load-use stall 和分支 flush
                benchmark_cycles_reg <= benchmark_cycles_reg + 32'd1;
            end
        end
    end
end

```

```
        end
    end

    assign benchmark_clear_ack = benchmark_clear_ack_reg;
    assign benchmark_done      = benchmark_done_reg;
    assign benchmark_running   = benchmark_running_reg;
    assign benchmark_cycles    = benchmark_cycles_reg;
```